PP Past papers, answered

All four papers (2022 to 2025), every part, answered at the length its marks need.

INTE 22313
OPEN BOOK

How to use this booklet with the exam note

- If a question in the exam looks like one of these, **find the closest past question here** and adapt the answer to the new scenario. The thumb index on the right goes 22, 23, 24, 25.
- The answers use the same wording as the note, so the two documents agree. For the theory behind a pattern, open that pattern's tab in the note.
- Each answer is **sized to its marks**: a 2-mark part gets a name and a reason; a 10-mark part gets a full diagram.
- Grey boxes hold the question as it was printed. The black tag is the part number and the right side shows the marks.

Index of every question

PAPER	Q	SCENARIO	PATTERNS / TOPIC
2022	1	Travel booking: air ticket, hotel, taxi from third-party services	Facade + Adapter
	2	Supermarket pay for executives, daily workers, temporary staff	Template Method (+ Factory)
	3	Email spam filter with sub-filters	Chain of Responsibility, HA & DR
	4	Cake shop · product framework · payroll one user · LMS pages · LMS modules	Decorator · Factory · Singleton · Proxy · Factory
2023	1	Salary components, executive deviations, configurable parameters	Template Method, config, Singleton
	2	Supermarket chain: purchasing, sales, warehousing	Microservices
	3	Restaurant coffee flavours, event websites, logging	Decorator · Adapter · Singleton, HA & DR
	4	Fantasy cricket · bank accounts · exceptions · LMS pages · authorization	Multiton · Factory · Chain · Proxy · Proxy
2024	1	Coffee shop add-ons, profit, logger	Decorator + Singleton (code)
	2	Library management system	Microservices
	3	Document workflow: Draft, Moderation, Published, Archived	State (state + class diagram)
	4	E-commerce checkout steps, exceptions, authorization	Template · Chain · Proxy
2025	1	Self-checkout kiosk	State · Decorator · Singleton (code)
	2	Media streaming platform	Microservices
	3	Library borrowing workflow	Chain · Factory · Proxy · Decorator (code)
	4	E-commerce orders and payments	Template · Chain · Singleton · Factory (code)

Every paper: 4 questions × 25 marks, answer all, 2½ hours, open book, "state any assumptions you make". So start each answer with one or two assumption lines, for example: "Assume the system is written in Java and third-party services can't be modified."

22 2022 · Question 1

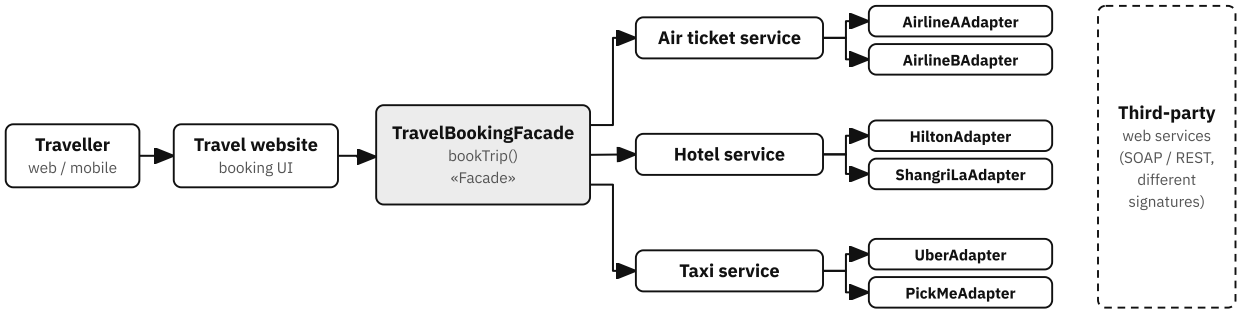
Travel booking website: air tickets, hotels and taxis from third-party web services.

FACADE + ADAPTER
25 MARKS

QUESTION 1 · 25 MARKS

Develop a website for travellers to book foreign trips. Users need to book an air ticket, a hotel and a taxi. When the user requests them, the system collects data from different web services and sends the details to the user. The Hotel sub-system connects to web services provided by different hotels (third party); the Air ticket sub-service connects to airline web services; the Taxi sub-service connects to taxi web services. These web services were developed by third parties, so their method signatures can differ.

1.1 *Draw a diagram to depict the above scenario.* 5 marks



Label the facade and the adapters. That shows the examiner where your patterns sit before you name them.

1.2 *What design pattern can be employed for the sub-systems that combine air tickets, hotels and taxis? Explain your reasons.* 4 marks

I would use the **Facade** pattern (structural). It provides one unified, simpler interface to a set of sub-systems.

A `TravelBookingFacade` with one method, `bookTrip(request)`, calls the AirTicketService, HotelService and TaxiService in the right order and returns one combined itinerary. **Reasons:** (1) the traveller does one action instead of three; (2) the website depends on **one class** instead of three sub-systems, so coupling is low; (3) the call order and the combining of results are written **once**; (4) if a sub-system changes, only the facade changes.

1.2.1 *Challenges with this pattern?* 3 marks

- The facade can grow into a **God class** as more services (car hire, insurance) are added.
- It is a **single point of failure**: if the facade or one service is down, the whole booking fails.
- **Partial bookings**: the flight is booked but the hotel fails.

1.2.2 *Mechanism to overcome them?* 3 marks

- Split into smaller facades (FlightFacade, StayFacade); the facade only delegates.
- Run the facade as stateless copies behind a load balancer; timeouts, retries and a fallback provider for each service.
- Compensating actions: cancel the flight if the hotel fails, or reserve all three first and confirm together.

22 2022 · Question 1 (cont.)

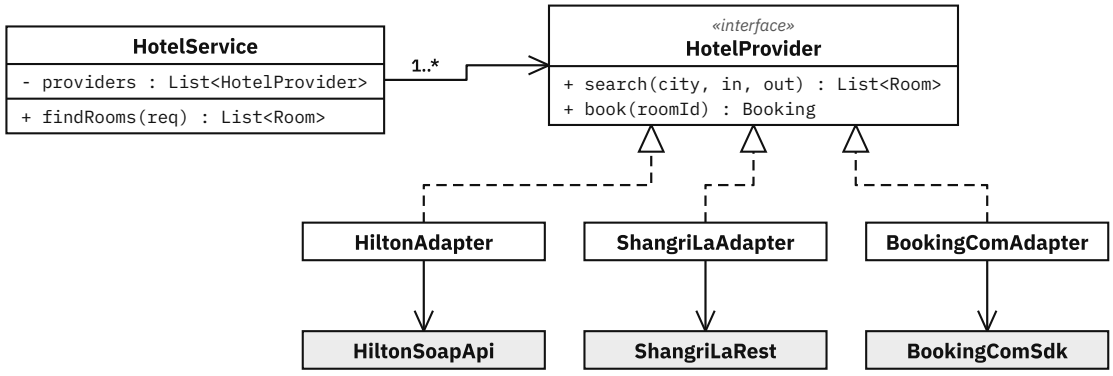
Connecting each sub-system to third-party web services with different signatures.

ADAPTER
10 MARKS

1.3 *What design patterns can be employed for connecting the sub-system with different third-party web services?* 4 marks

I would use the **Adapter** pattern (structural). It converts the interface of a class into another interface the client expects, so classes with incompatible interfaces can work together.

The HotelService owns one interface, `HotelProvider` (`search()`, `book()`). Each hotel's web service gets its own adapter (`HiltonAdapter`, `ShangriLaAdapter` …) that implements `HotelProvider` and translates the call into that hotel's own SOAP / REST / SDK method. Airlines and taxis are done the same way. **Reasons:** the third-party services **can't be modified**; the sub-system code stays the same whichever provider is used; a **new provider is just a new adapter** (Open/Closed).



1.3.1 *Challenges with this pattern?* 3 marks

- **Many adapters** to write and maintain, one per provider.
- **Different data formats** (XML vs JSON, currencies, date formats) and error codes.
- A provider **changes its API version**, or is slow or unavailable.

1.3.2 *Mechanism to overcome them?* 3 marks

- One common interface; a **Factory or registry** picks the adapter from configuration.
- A common data object (DTO) and a **mapper inside each adapter**; unit-test every adapter.
- Only that adapter changes when an API changes (version adapters); **timeouts, retries, caching** and fall back to another provider.

Adapter code for one provider

not asked, but adds weight to 1.3

```
public interface HotelProvider {                // Target
    List<Room> search(String city, LocalDate in, LocalDate out);
}
public class HiltonAdapter implements HotelProvider {  // Adapter
    private final HiltonSoapApi api = new HiltonSoapApi(); // Adaptee
    public List<Room> search(String city, LocalDate in, LocalDate out) {
        String xml = api.findRooms(city, in.toString(), out.toString());
        return RoomMapper.fromHiltonXml(xml);   // translate the result
    }
}
```

22 2022 · Question 2

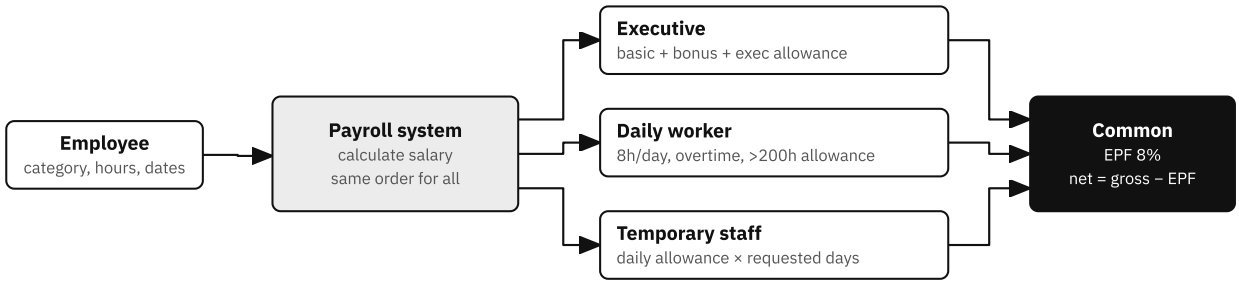
Supermarket pay calculation for different employee categories.

TEMPLATE METHOD
25 MARKS

QUESTION 2 · 25 MARKS

A supermarket has different pay calculations for executive employees, temporary staff and daily workers. **Executives:** basic salary, bonus and executive allowances. **Daily workers:** 8 hours per day; extra hours earn overtime; more than 200 hours earns an attendance allowance. **Temporary staff:** called when required and paid a daily allowance for the requested dates. All employees contribute 8% to the Employee Provident Fund.

2.1 *Draw a diagram to depict the above scenario.* 4 marks



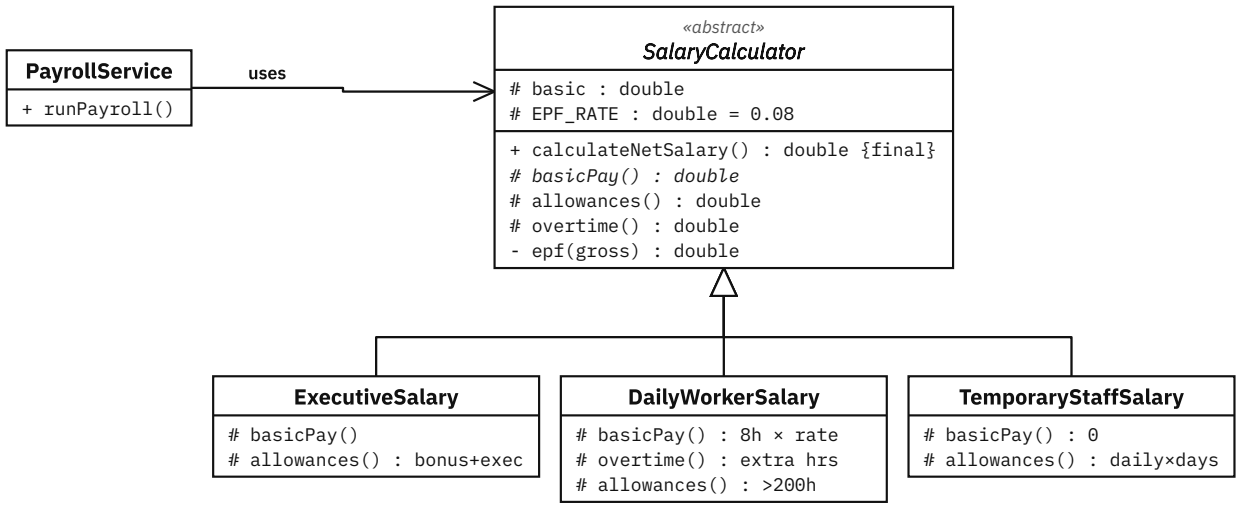
2.2 *What design pattern(s) can be employed? Explain the reasons for your selection.* 6 marks

I would use the **Template Method** pattern (behavioural). It defines the skeleton of an algorithm in a superclass and lets subclasses redefine certain steps without changing the structure.

Every category follows the **same calculation order**: work out the earnings → gross pay → deduct EPF 8% → net pay. The EPF deduction and the net calculation are **the same for everyone**; only the earning components differ (bonus, overtime, attendance allowance, daily allowance). So an abstract `SalaryCalculator` has a final `calculateNetSalary()` that fixes the order and does the common parts, and each category overrides only its own components.

Reasons: (1) the common EPF and net logic is written **once**; (2) every category is calculated in the **same, enforced order**, so results are consistent; (3) a new category (e.g. part-time) is **one new subclass** (Open/Closed). **Supporting patterns:** a **Factory** creates the right calculator from the employee's category, and the payroll run can be a **Singleton** so it runs once at a time.

2.3 *Draw a class diagram for the above design pattern.* 5 marks




```
public final double calculateNetSalary() {            // template method
    double gross = basicPay() + allowances() + overtime();
    return gross - gross * 0.08;                      // EPF 8%, same for everyone
}
```

2.4 *What challenges will you face?* 3 marks

- The order of steps is **rigid**: a category that needs a different sequence doesn't fit.
- The number of **subclasses grows** with every new category.
- Rates and limits (8%, 200 hours, 8 hours/day) **change over time**.
- A subclass might **override a common step** by mistake.

2.5 *Mechanism to overcome them?* 3 marks

- **Hook methods** with default 0 for optional parts (bonus, overtime), so subclasses override only what they need.
- Shared middle classes (e.g. `HourlyPaid`) for categories that share rules.
- Read rates from **external configuration** through a Singleton ConfigManager.
- Make the template method **final** and the common steps private.

2.6 *"Code duplication can be achieved from the used design pattern in the above scenario." Do you agree? Explain.* 4 marks

I disagree. Template Method is used to **avoid** code duplication. The parts every employee shares (the order of steps, the EPF 8% deduction, the net salary formula) are written once in the abstract `SalaryCalculator`. Each subclass contains only what is different for its category. Without the pattern, each category would copy the whole calculation and the EPF logic.

A small amount of duplication can remain if two subclasses override a step in the same way (e.g. two categories with the same overtime rule). The fix is to move that shared code up into the parent or into a shared middle class.

22 2022 · Question 3

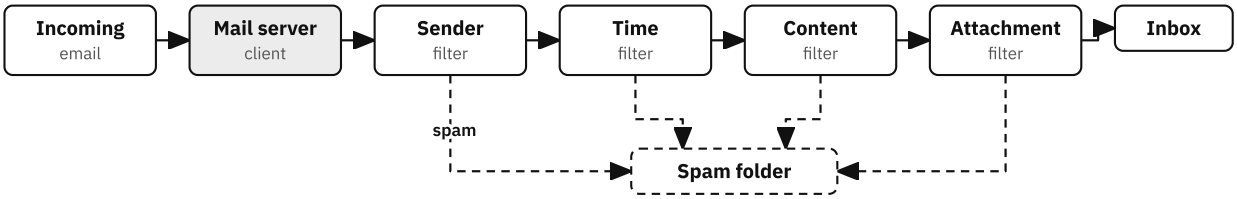
Email spam filter made of sub-filters.

CHAIN OF RESPONSIBILITY
25 MARKS

QUESTION 3 · 25 MARKS

Design an email spam filter. Multiple sub-spam filters are available depending on the different components of the email, such as the email sender, the time it was sent, its content and its attachments.

3.1 *Sketch a diagram and explain the different components of the email spam filter.* 4 marks

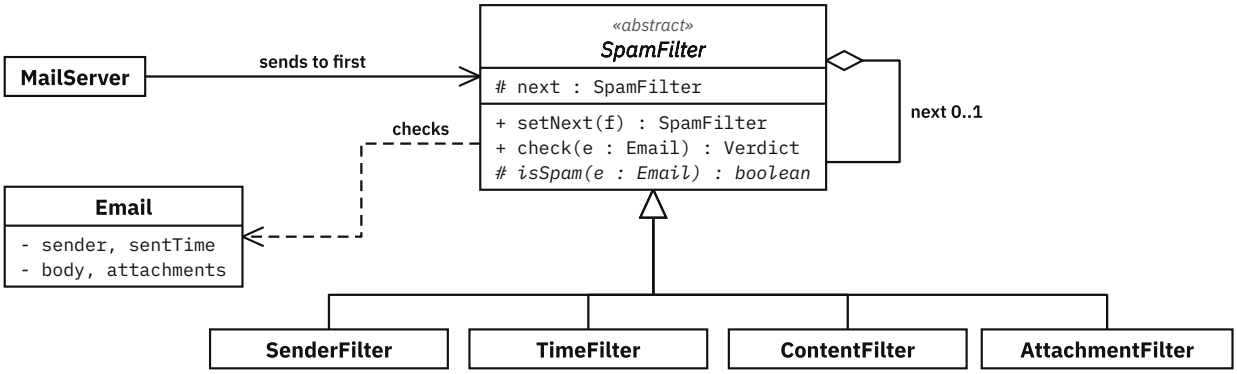


The **mail server** receives the email and passes it to the first filter. The **sender filter** checks blacklists and domain reputation. The **time filter** flags odd sending hours and bursts. The **content filter** checks keywords, links and language patterns. The **attachment filter** checks file types and scans for viruses. Any filter can mark the email as spam and stop; if all pass, it is delivered to the inbox.

3.2 *What design pattern can you follow? Explain the reasons.* 5 marks

I would use the **Chain of Responsibility** pattern (behavioural). It avoids coupling the sender of a request to its receivers by giving more than one object a chance to handle it; the handlers are chained and the request is passed along.

Each sub-filter is a handler that extends an abstract `SpamFilter` and holds a link to the next filter. **Reasons:** (1) the mail server only knows the first filter, so it is **decoupled** from the filters; (2) each filter has **one responsibility**; (3) filters can be **added, removed or reordered** without changing the others; (4) the chain can be built from configuration at runtime.



2022 · Question 3 (cont.)

22

Prioritising the filters, extending the chain, and High Availability / Disaster Recovery.

CHAIN OF RESPONSIBILITY
25 MARKS

3.3 *How do you prioritise the different components? Explain your prioritisation criteria.* 6 marks

CRITERION	WHAT IT MEANS HERE
Cost / speed	Cheap checks first (sender blacklist, sent time take microseconds); expensive ones last (content analysis, virus scanning of attachments). Most spam is stopped before the expensive filters run.
Rejection rate	The filter that catches the most spam goes early, saving work for the others.
Severity / security	A dangerous attachment (malware) may be moved earlier because the risk is high.
Accuracy	Filters with few false positives come before risky ones, so good mail isn't lost.

Resulting order: Sender → Time → Content → Attachment. The order is kept in configuration so it can be tuned using real statistics.

3.4 *What would be your suggestion if more sub-components need to be added?* 5 marks

Write a new class that extends `SpamFilter` and implements only its own check, for example `LinkReputationFilter` or `LanguageFilter`. Insert it into the chain with `setNext()`, or list it in the configuration file that builds the chain. The mail server and the existing filters **don't change**, which follows the Open/Closed principle. Each new filter is unit-tested on its own, and a fallback "deliver to inbox" handler stays at the end.

```
class LinkReputationFilter extends SpamFilter {
    protected boolean isSpam(Email e) { return LinkDb.hasBadLink(e.getBody()); }
}
sender.setNext(time).setNext(content).setNext(new LinkReputationFilter()).setNext(attachment);
```

3.5 *How can you extend this design to achieve High Availability (HA) and Disaster Recovery (DR)?* 5 marks

HA: the filters don't keep state, so the whole chain can run on **several servers behind a load balancer**, with health checks and automatic restart. Incoming mail goes into a **message queue** so nothing is lost if a server dies. If one filter times out (e.g. the virus scanner), a backup handler skips or retries it instead of blocking all mail.

DR: blacklists, rules and the configuration are **backed up and replicated to a second region**. A defined RPO/RTO, automatic failover to the DR site, and regular DR drills.

22 2022 · Question 4

Five short scenarios. Name the pattern, explain it, give two reasons.

5 × 5 MARKS
25 MARKS

4.1 *Online cake ordering: users select options such as decorations and colour. Prices and delivery times differ depending on the selections.* — 5 marks

Decorator (structural). A `Cake` interface has `getPrice()` and `getDeliveryDays()`. `PlainCake` is the base. Each option (`FondantDecoration`, `ChocolateColour`, `FlowerTopping`) is a decorator that wraps the cake and adds its own price and extra delivery days. **Reasons:** any mix of options is built at runtime without a subclass per combination (no class explosion), and a new option is just a new decorator.

`Cake c = new FlowerTopping(new Fondant(new PlainCake()));` → price = 2,500 + 800 + 600; days = 1 + 1 + 0.

4.2 *Develop a scalable and extensible product-based framework for future needs of the cake ordering system.* — 5 marks

Factory Method (creational). A `Product` interface (cakes, cupcakes, gift boxes…) and a `ProductFactory.create(type)` that returns the right product. Client code works only with `Product`. **Reasons:** new product types are added as new classes registered with the factory, so the framework grows without changing existing code (Open/Closed), and creation logic is kept in one place. The Decorator from 4.1 can still wrap whatever the factory returns.

4.3 *Payroll system: salary calculation should be run by only one user, and all other connections to the salary system stopped.* — 5 marks

Singleton (creational) with a lock. `SalaryCalculationManager` has a private constructor and a static `getInstance()`, so only one calculation engine exists. Its `runPayroll(user)` is `synchronized` and sets a `running` flag; while it is set, other logins and connections are refused or queued. **Reasons:** one controller means no two payroll runs at once (no double payments or inconsistent data), and access is controlled in one place. Note that the Singleton limits objects; the lock is what blocks other users.

4.4 *LMS: when users read large content, load only a couple of pages rather than the entire document.* — 5 marks

Proxy, a **virtual proxy** (structural). `DocumentProxy` implements the same `Document` interface as `LargeDocument`. `getPage(n)` loads only the requested page (and maybe the next) and caches it; the full document is never loaded unless needed. **Reasons:** fast opening, low memory and bandwidth, and the reader code doesn't change because the interface is the same.

4.5 *An LMS that needs different modules to be created depending on the user's needs.* — 5 marks

Factory Method (creational). A `Module` interface implemented by `QuizModule`, `ForumModule`, `AssignmentModule`, `VideoModule`. `ModuleFactory.create(userNeed)` returns the right one. **Reasons:** the LMS doesn't need to know concrete module classes (loose coupling), and new modules are added without changing the existing code.

2023 · Question 1

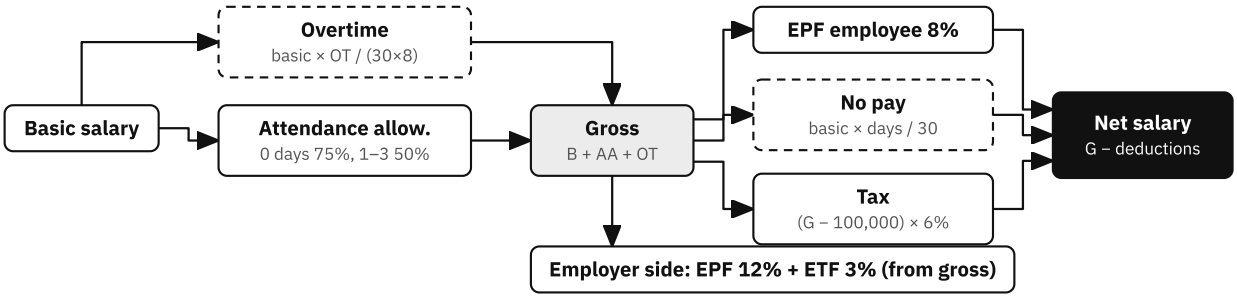
23 Salary components with deviations for executives, configurable parameters, one-user payroll run.

TEMPLATE + SINGLETON
25 MARKS

QUESTION 1 · 25 MARKS

Overtime = Basic × OT hours / (30 × 8) · Attendance allowance: absent > 3 days → none; 1–3 days → 50% of basic; 0 days → 75% of basic · Gross = Basic + Attendance + Overtime · EPF employee = Gross × 0.08 · EPF employer = Gross × 0.12 · ETF = Gross × 0.03 · No pay = Basic × no-pay days / 30 · Tax = (Gross − 100,000) × 0.06 · Total deduction = EPF employee + No pay + Tax · Net = Gross − Total deduction · Total EPF = EPF employee + EPF employer.
Permanent employees use all of these; for **executive** employees there is **no overtime and no no-pay**.

1.1 *Draw a diagram to indicate the above scenario.* 4 marks



1.2 *What design pattern can be employed for the sub-systems that calculate different salary components for different employees? Reasons.* 5 marks

I would use the **Template Method** pattern (behavioural). It defines the skeleton of an algorithm in a superclass and lets subclasses redefine certain steps without changing the structure.

The calculation always runs in the same order: overtime and allowance → gross → deductions → net (dashed boxes in 1.1 are 0 for executives). An abstract `SalaryCalculator` has a final `calculateNetSalary()` containing that order and every common formula. `overtime()`, `attendanceAllowance()` and `noPay()` are hook methods. `PermanentSalary` uses the standard formulas; `ExecutiveSalary` overrides `overtime()` and `noPay()` to return 0. **Reasons:** shared formulas exist **once**; the **same order** is enforced; a new type is **one subclass**.

The design in code

supports 1.2, write it if time allows

```
public abstract class SalaryCalculator {
  protected double basic;
  protected int otHours, absentDays,
                noPayDays;

  public final double calculateNetSalary() {
    double gross = basic
        + attendanceAllowance() + overtime();
    double epfEmp = gross * cfg("epf.emp");
    double taxable =
        gross - cfg("tax.threshold");
    double tax = Math.max(0,
        taxable * cfg("tax.rate"));
    return gross - (epfEmp + noPay() + tax);
  }

  // hooks with the standard formulas
  protected double overtime() {
    return basic * otHours / (30 * 8); }
  protected double noPay() {
    return basic * noPayDays / 30; }
  protected double attendanceAllowance() {
    if (absentDays == 0)
      return basic * 0.75;
    if (absentDays <= 3)
      return basic * 0.50;
    return 0; }
}
class PermanentSalary
    extends SalaryCalculator {}
class ExecutiveSalary
    extends SalaryCalculator {
  protected double overtime() { return 0; }
  protected double noPay()    { return 0; }
}
```

2023 · Question 1 (cont.)

23

The code, challenges, configurability and a payroll run that blocks everyone else.

TEMPLATE + SINGLETON
25 MARKS

1.3 *Challenges with the selected pattern?* 4 marks

- The **order is fixed**; an employee type that needs a different sequence doesn't fit.
- **Rates and thresholds change** (8%, 12%, 3%, 100,000, 6%), and hard-coded values need a redeploy.
- **Subclasses multiply** as new employee types appear.
- A subclass may **override a common formula** by mistake.

1.4 *Mechanism to overcome them?* 4 marks

- **Hooks** with default bodies for optional components; Strategy if a whole step must vary.
- All rates in **external configuration** (1.5).
- Shared middle classes for types with common rules.
- Template method **final**; common formulas private.

1.5 *Many of the parameters can change over time. How would you improve configurability? State the configuration.* 4 marks

Move every rate and threshold out of the code into **external configuration**, a database table or a properties file. A **Singleton ConfigManager** loads it once; the calculator reads values through `cfg(key)`. An admin screen updates them; each change is **versioned with an "effective from" date** and audited, so old payslips can be recalculated with the old rates. No recompile or redeploy is needed.

```
# salary.properties
ot.divisor.days=30
ot.divisor.hours=8
attendance.full=0.75
attendance.partial=0.50
attendance.maxAbsent=3
epf.emp=0.08
epf.employer=0.12
etf=0.03
tax.threshold=100000
tax.rate=0.06
```

1.6 *During salary calculation no other user should be logged in and no other process should run. Which pattern and why?* 4 marks

Singleton, combined with a lock. `PayrollEngine` has a private constructor and a static `getInstance()`, so there is **only one calculation controller** in the system. Its `synchronized runPayroll()` sets a `running` flag; the login service and other processes check `PayrollEngine.getInstance().isRunning()` and are refused or queued until it finishes. **Why:** one controlling object means two calculations can't run at the same time, and the lock gives one place that blocks everyone else, so the data stays consistent.

23 2023 · Question 2

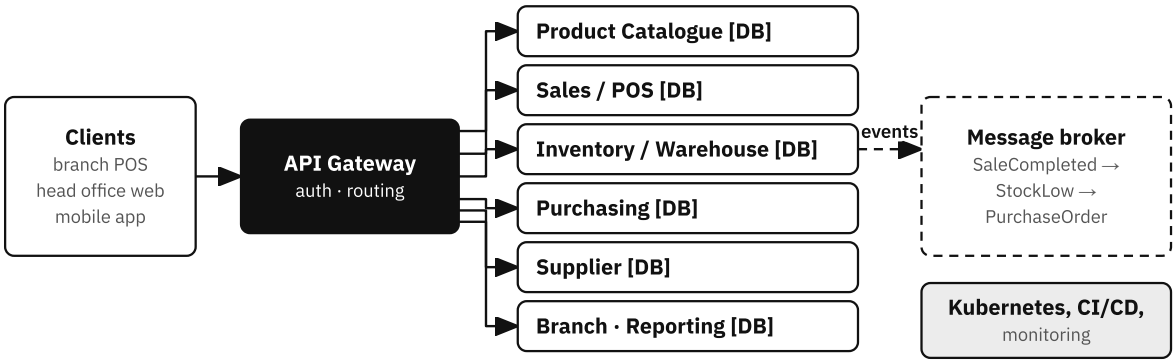
A supermarket that is part of a chain: purchasing, sales, warehousing.

MICROSERVICES
25 MARKS

QUESTION 2 · 25 MARKS

A supermarket has functions such as purchasing, sales and warehousing. You are hired to design a system for it. The supermarket is part of a supermarket chain.

2.1 *Draw a diagram to depict the above scenario.* 4 marks



2.2 *How can an organisation effectively implement and manage a microservices architecture to achieve scalability, fault isolation and ease of continuous deployment?* 5 marks

Scalability	Each service scales on its own. Sales/POS gets extra copies at weekends and in the festive season; Purchasing doesn't. Stateless services on Kubernetes with auto-scaling, plus a cache for the product catalogue.
Fault isolation	Each service has its own process and database. If Reporting crashes, branches can still sell. Circuit breakers, timeouts and retries; POS keeps working offline and syncs later.
Continuous deployment	Own repository and CI/CD pipeline per service, automated tests, Docker images, blue-green or canary releases, versioned APIs. One team per service.
Manage it	API gateway, service registry, central config, and central logging, metrics and tracing.

23 2023 · Question 2 (cont.)

Architecture, data consistency, security, and identical platforms.

MICROSERVICES
25 MARKS

2.3 *Propose a microservice architecture for the above problem.* 4 marks

SERVICE	OWNS
Product Catalogue	items, prices, promotions (shared by every branch)
Sales / POS	bills, payments at the till, returns
Inventory / Warehouse	stock per branch and per warehouse, transfers between branches
Purchasing	purchase orders, reorder levels, goods received
Supplier	supplier details, contracts, invoices
Branch	branches of the chain, staff, opening hours
Customer / Loyalty, Reporting, Notification	points, chain-wide sales reports, SMS/email alerts

Each service has its own database. Clients (POS terminals, head office web, mobile app) go through an **API gateway**. Services publish events to a **message broker** ("SaleCompleted" → Inventory reduces stock → Purchasing reorders when below the level). Everything runs in containers on Kubernetes.

2.4 *Methods to maintain data consistency in the proposed architecture?* 4 marks

- **Database per service**, so there are no shared transactions; accept **eventual consistency**.
- **Saga pattern**: a sale reduces stock; if the payment fails, a **compensating action** puts the stock back. Coordinated by events or by an orchestrator.
- **Event-driven updates** through Kafka/RabbitMQ, with a **transactional outbox** and **idempotent consumers** so a repeated event does no harm.
- **CQRS** read models for chain-wide reports. Two-phase commit only where it is essential.

2.5 *How would you achieve security?* 4 marks

- **OAuth2 / OpenID Connect** at the gateway with **JWT**; roles for cashier, branch manager, buyer, head office (RBAC).
- **Mutual TLS** between services; least-privilege database users.
- HTTPS everywhere, card data handled to **PCI-DSS**, secrets in a vault.
- Rate limiting, input validation, audit logs.

2.6 *Why is it essential to provide the same hardware configurations and options for each microservice platform?* 4 marks

Identical environments (containers and infrastructure-as-code) make **deployment, scaling and failover predictable**: any node can run any service, and performance is the same everywhere. **Capacity planning, automation, monitoring and troubleshooting** are simpler, it avoids "works on my machine", and a failed node can be **rebuilt exactly** for disaster recovery. (Exception: special nodes such as GPUs for ML.)

2023 · Question 3

23 Restaurant with many coffee flavours, event websites from other vendors, logging, HA and DR.

DECORATOR · ADAPTER · SINGLETON
25 MARKS

QUESTION 3 · 25 MARKS

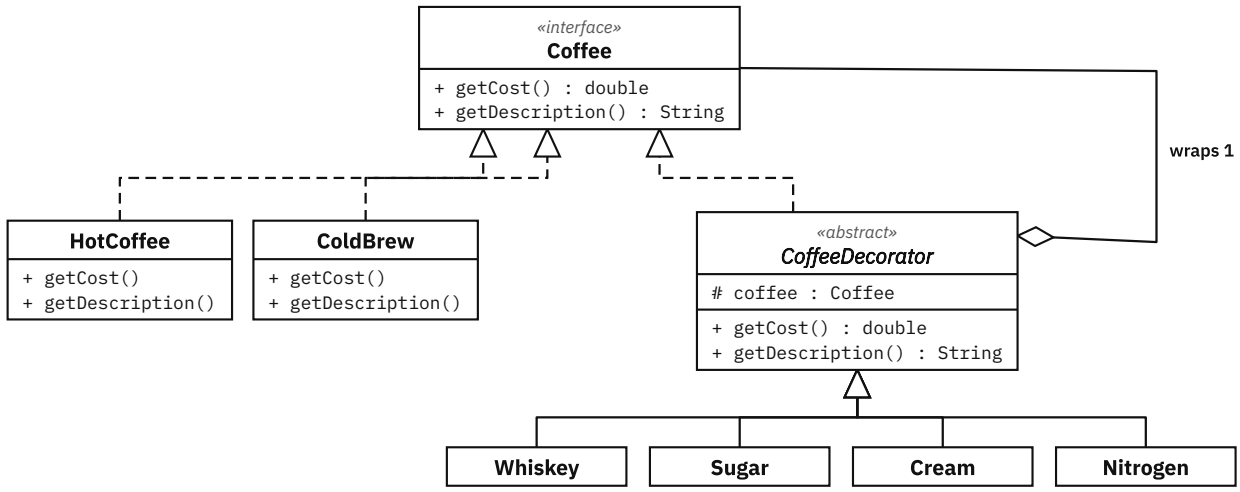
A restaurant has different flavours of coffee. For example, Irish Coffee is hot coffee, Irish whiskey and sugar, topped with cream; Nitro Coffee is cold-brew coffee infused with nitrogen, served on tap, with a creamy texture.

3.1 *Considering the many combinations that exist for coffee, what is the proposed design pattern?* 5 marks

I would use the **Decorator** pattern (structural). It attaches additional responsibilities to an object dynamically by wrapping it, which is a flexible alternative to subclassing.

The base coffees (`HotCoffee`, `ColdBrew`) are concrete components. Every ingredient (Whiskey, Sugar, Cream, Nitrogen) is a decorator that wraps a coffee and adds its own price and description. Irish Coffee = `new Cream(new Sugar(new Whiskey(new HotCoffee())))`; Nitro Coffee = `new Nitrogen(new ColdBrew())`. **Reasons:** the number of combinations is huge (n ingredients → 2^n subclasses), so wrapping avoids class explosion; new recipes are made at runtime; a new ingredient is one new class (Open/Closed).

3.2 *Draw a simple class diagram to facilitate the above design pattern.* 5 marks



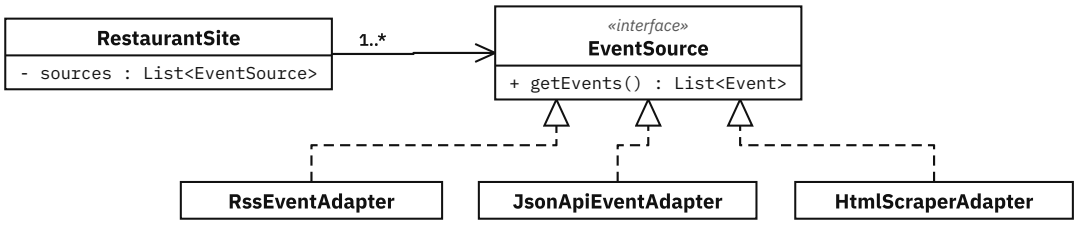
23 2023 · Question 3 (cont.)

Integrating other vendors' websites, logging, and HA / DR.

ADAPTER · SINGLETON
25 MARKS

3.3 *The restaurant must connect to different websites to show events organised by different entities. They were developed with different technologies by different vendors. How do you integrate them?* 5 marks

Use the **Adapter** pattern (structural). The restaurant system owns one interface, `EventSource` with `getEvents()`, returning a common `Event` object (title, date, venue). Each website gets its own adapter (`RssEventAdapter`, `JsonApiEventAdapter`, `HtmlScraperAdapter`) that calls that site in its own technology and converts the result into `Event`. The restaurant site just loops over a list of `EventSource`. **Reasons:** the vendors' systems can't be changed; the restaurant code doesn't depend on any vendor; a new vendor is just a new adapter. A **Facade** (`EventFeed.getAllEvents()`) can then merge all sources into one list.



3.4 *If you must implement a logging mechanism, which design pattern can you use? Give logical reasons.* 5 marks

Use the **Singleton** pattern for the logger. **Reasons:** (1) all parts of the system must write to **the same log file**; several logger objects would open the file several times and produce duplicate or interleaved lines; (2) one instance owns the file handle, so the format and order are consistent; (3) a global access point (`Logger.getInstance()`) from anywhere; (4) less memory. `log()` is synchronized so threads don't mix lines. (Adding logging around existing services without changing them can be done with a logging **Proxy** that calls this Singleton.)

```
public final class Logger {
  private static final Logger INSTANCE =
      new Logger();
  private final PrintWriter out;
  private Logger() {
    out = Files.openAppend("app.log"); }
  public static Logger getInstance() {
    return INSTANCE; }
  public synchronized void log(String m) {
    out.println(LocalDateTime.now()
        + " " + m);
  }
}
```

3.5 *How can you extend this design to achieve High Availability (HA) and Disaster Recovery (DR)?* 5 marks

HA: run the ordering application as **several stateless copies behind a load balancer**, with health checks and auto-restart; the database is replicated with automatic failover. Event adapters use timeouts, retries and a **cache** of the last events, so a vendor site being down doesn't break the page. A single logger instance per server ships logs to a central, replicated log store, so the Singleton isn't a single point of failure. **DR:** regular backups, the database replicated to a **second region**, agreed RPO/RTO, automated failover, infrastructure-as-code to rebuild, and DR drills.

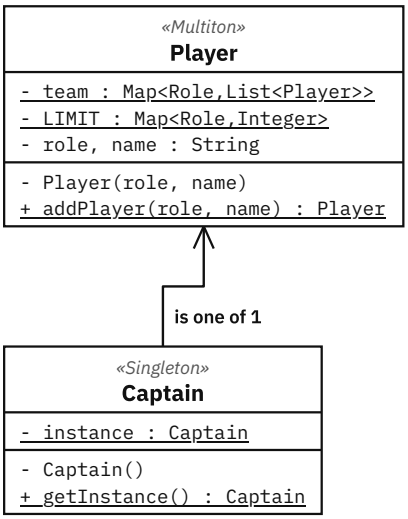
23 2023 · Question 4

Five scenarios. Pattern, explanation and a simple class diagram for each.

5 × 5 MARKS
25 MARKS

4.1 *Fantasy cricket: the user selects a maximum of five batsmen, five bowlers and one wicketkeeper; there should be one captain. How would you use an extended Singleton to achieve this?* 5 marks

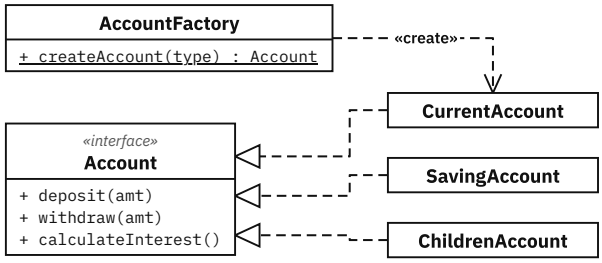
An extended Singleton is a **Multiton**: the class controls how many objects can exist, but the limit is N per key instead of 1. `Player` has a private constructor; the only way to get one is `addPlayer(role, name)`, which counts the players of that role and throws an exception once the limit (5 batsmen, 5 bowlers, 1 wicketkeeper) is reached. `Captain` is a plain Singleton (exactly one).




```
public static synchronized Player addPlayer(
        String role, String name) {
  team.putIfAbsent(role, new ArrayList<>());
  int n = team.get(role).size();
  if (n >= LIMIT.get(role))
    throw new IllegalStateException(
        "Max " + role + " reached");
  Player p = new Player(role, name);
  team.get(role).add(p);
  return p;
}
```

4.2 *A banking system with different types of accounts: Current, Saving and Children accounts.* 5 marks

Factory Method (creational). An `Account` interface with `deposit()`, `withdraw()`, `calculateInterest()`. `CurrentAccount` (overdraft, no interest), `SavingAccount` (interest) and `ChildrenAccount` (withdrawal limit, guardian) implement it. `AccountFactory.createAccount(type)` returns the right one. **Reasons:** the bank code works only with `Account` (loose coupling), creation logic is in one place, and a new account type (FixedDeposit) is added without changing existing code.



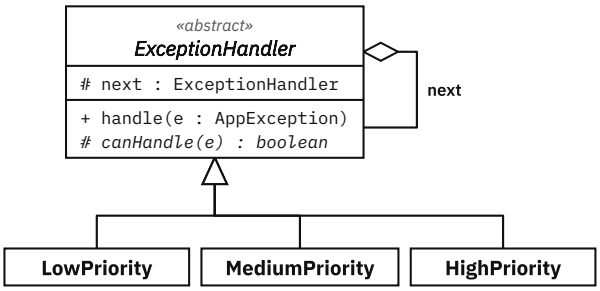
23 2023 · Question 4 (cont.)

Exceptions by priority, loading part of a document, and authorization options.

CHAIN · PROXY
15 MARKS

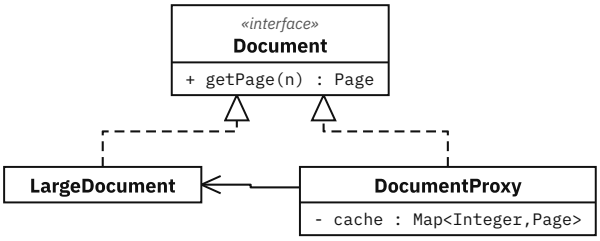
4.3 *An exception-handling system that can handle different priorities depending on the problem.* 5 marks

Chain of Responsibility (behavioural). Each handler deals with the exceptions of its own priority and passes the rest to the next: `LowPriorityHandler` (validation errors: show a message) → `MediumPriorityHandler` (payment or timeout errors: retry) → `HighPriorityHandler` (system failures: log and alert the admin) → `DefaultHandler` (generic error page). **Reasons:** the code that throws doesn't need to know who handles it; handlers can be added or reordered; it works like nested try/catch blocks.



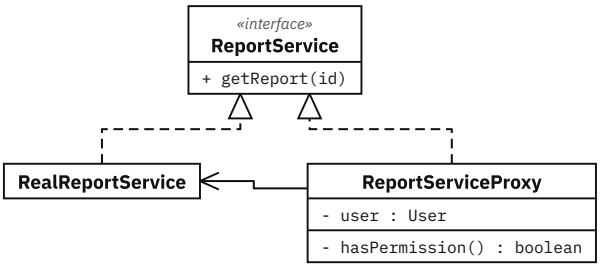
4.4 *LMS: when users read large content, load only a couple of pages rather than the entire document.* 5 marks

Proxy, a **virtual proxy** (structural). `DocumentProxy` and `LargeDocument` both implement `Document`. The reader holds the proxy; `getPage(n)` loads only that page from storage and caches it, and the full document is never loaded unless needed. **Reasons:** fast opening, low memory and bandwidth; same interface, so the reader code doesn't change.



4.5 *A website with different authorization options, depending on the user's need.* 5 marks

Proxy, a **protection proxy** (structural). Each protected service (`ReportService`, `AdminService`) sits behind a proxy with the same interface. The proxy checks the user's role and permissions, logs the access and only then forwards the call. **Reasons:** security is in one place and the real services stay clean; access rules change without touching the services. If authorization is a series of checks (login → role → permission → 2FA), combine with a **Chain of Responsibility**.



2024 · Question 1

24 Coffee shop with optional ingredients, different profit per coffee, and one shared logger.

DECORATOR + SINGLETON
25 MARKS

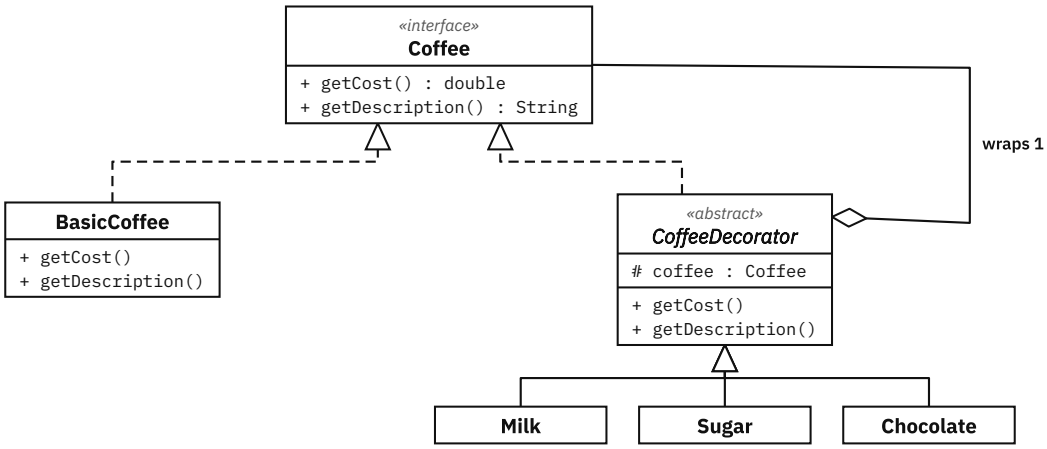
QUESTION 1 · 25 MARKS

In a coffee shop, customers order a base coffee and add optional ingredients (milk, sugar, chocolate). The price depends on the base type and the selected additions. You need a flexible way to calculate the price dynamically without a complex hierarchy of subclasses for every combination of coffee and ingredients.

1.1 *What is the best design pattern for the above scenario?* 5 marks

The **Decorator** pattern (structural). It attaches additional responsibilities to an object dynamically by wrapping it, which is a flexible alternative to subclassing.

The base coffee is a concrete component; Milk, Sugar and Chocolate are decorators. Each decorator implements the same `Coffee` interface, holds the coffee it wraps, adds its own price and description, and passes the rest of the call inwards. **Reasons:** (1) no class explosion: 3 ingredients would need $2^3 = 8$ subclasses per base coffee, here it's one class per ingredient; (2) the price is worked out **at runtime** from whatever the customer picks; (3) a new ingredient is a new class and nothing existing changes (Open/Closed); (4) each decorator has one responsibility.



1.2 *Define a Coffee interface with getCost() and getDescription().* 5 marks

```
// Component: every coffee and every
// decorator has these two methods
public interface Coffee {
    double getCost();
    String getDescription();
}
```

1.3 *Create a BasicCoffee class implementing the Coffee interface.* 5 marks

```
// Concrete component: the base coffee
public class BasicCoffee implements Coffee {
    public double getCost() {
        return 200.0;
    }
    public String getDescription() {
        return "Basic coffee";
    }
}
```

2024 · Question 1 (cont.)

24 The decorators that make 1.2 and 1.3 work, the profit extension, and the logger.

DECORATOR + SINGLETON
25 MARKS

Show the decorators too

adds strength to 1.1–1.3

```
public abstract class CoffeeDecorator
        implements Coffee {
    protected Coffee coffee;    // wrapped
    public CoffeeDecorator(Coffee c) {
        coffee = c; }
    public double getCost() {
        return coffee.getCost(); }
    public String getDescription() {
        return coffee.getDescription(); }
}
```

```
public class Milk extends CoffeeDecorator {
    public Milk(Coffee c) { super(c); }
    public double getCost() {
        return coffee.getCost() + 80; }
    public String getDescription() {
        return coffee.getDescription()
               + ", milk"; }
}
// Sugar +20, Chocolate +120: same shape
Coffee c = new BasicCoffee();
c = new Sugar(new Milk(c));
c.getCost();   // 200 + 80 + 20 = 300
```

1.4 *If there is a different profit for each coffee, how would you extend the above design?* 5 marks

Option A: add `getProfit()` to the `Coffee` interface. Each base coffee returns its own margin; each decorator returns `coffee.getProfit()` plus its add-on profit. Selling price = `getCost() + getProfit()`.

Option B (better, Open/Closed): add a new `ProfitMargin` decorator that wraps the finished drink and adds a percentage. No existing class changes, and each coffee can get a different rate: `new ProfitMargin(espresso, 0.25)`, `new ProfitMargin(latte, 0.35)`. The rates can be read from configuration.

```
public class ProfitMargin
        extends CoffeeDecorator {
    private final double rate;
    public ProfitMargin(Coffee c, double r) {
        super(c);
        rate = r;
    }
    public double getCost() {
        return coffee.getCost() * (1 + rate);
    }
}
```

1.5 *A logger must write all logs to the same file or output stream; multiple instances would cause duplicate or inconsistent output. Propose a design pattern.* 5 marks

Use the **Singleton** pattern (creational). It ensures a class has only one instance and provides a global point of access to it. The Logger has a private constructor and a static `getInstance()`, so every class gets the **same** logger, which owns the **one** file handle. **Reasons:** no duplicate or interleaved lines, one consistent format and order, less memory, and easy access from anywhere. `log()` is `synchronized` so lines from different threads don't mix.

```
public final class Logger {
  private static volatile Logger instance;
  private final PrintWriter out;
  private Logger() {
    out = Files.openAppend("app.log"); }
  public static Logger getInstance() {
    if (instance == null) {
      synchronized (Logger.class) {
        if (instance == null)
          instance = new Logger();
      }
    }
    return instance;
  }
  public synchronized void log(String m) {
    out.println(m); }
}
```

24 2024 · Question 2

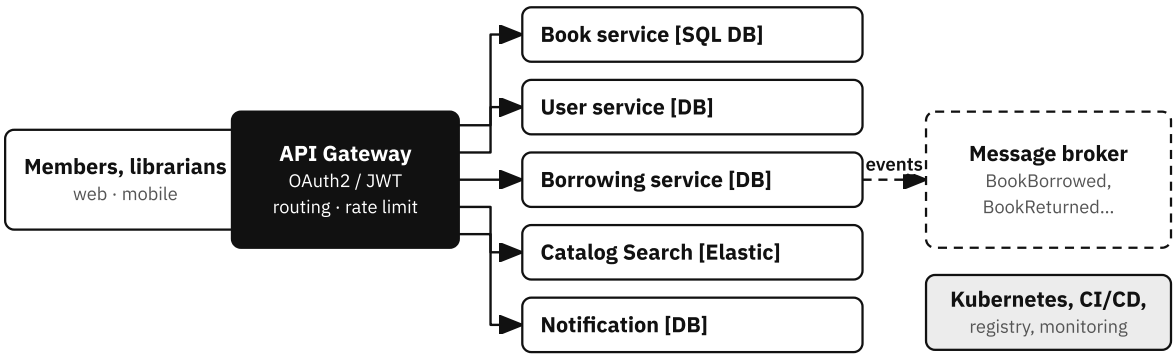
Library management system using microservices.

MICROSERVICES
25 MARKS

QUESTION 2 · 25 MARKS

Develop a scalable, maintainable library management system using microservices. Requirements: **Book management** (add, update, search by title, author, genre, ISBN) · **User management** (register, accounts, borrowing history) · **Borrowing and returning** (borrow, renew, return, due dates) · **Notifications** (due dates, new arrivals, system updates) · **Catalog search** (advanced filters, real-time availability).

2.1 *Draw a diagram to depict the above scenario.* 5 marks



2.2 *How can an organisation effectively implement and manage microservices to achieve scalability, fault isolation and ease of continuous deployment in this scenario?* 5 marks

Scalability	Scale each service on its own. **Catalog Search** gets the most traffic, so it gets more copies and a search index; Borrowing scales near semester start; User and Notification stay small. Stateless services, Kubernetes auto-scaling, a cache for popular searches.
Fault isolation	Each service has its own process and database. If Notification is down, borrowing still works and messages wait in the queue. Circuit breakers, timeouts, retries; search falls back to cached results.
Continuous deployment	Separate repository and CI/CD pipeline per service with automated tests, Docker images, blue-green or canary releases, and versioned APIs, so the Book service can be updated without touching Borrowing.
Manage it	API gateway, service registry, central config, and central logging, metrics and tracing.

24 2024 · Question 2 (cont.)

Architecture, data consistency and security for the library.

MICROSERVICES
25 MARKS

2.3 *Propose a microservice architecture for the above problem.* 5 marks

SERVICE	RESPONSIBILITY	DATA / TECH
Book	add / update books, copies, ISBN	own SQL DB
User	registration, accounts, membership	own DB
Borrowing	borrow, renew, return, due dates, fines	own DB
Notification	due-date reminders, new arrivals, updates (email/SMS)	consumes events
Catalog Search	advanced filters, real-time availability	Elasticsearch index built from Book and Borrowing events (CQRS)

Web and mobile clients go through an **API gateway**. Services call each other with REST when they need an answer now (Borrowing asks User "is the membership valid?") and publish events to **Kafka/RabbitMQ** otherwise ("BookBorrowed" → Search marks it unavailable, Notification schedules a reminder). Containers on Kubernetes, with a registry, config server and monitoring.

2.4 *Methods to maintain data consistency in the proposed architecture?* 5 marks

- **Database per service**, so no shared transactions; we accept **eventual consistency** (search may show a book as available for a second).
- **Saga** for borrowing: reserve the copy → record the loan → update the user's history. If a step fails, **compensating actions** undo the earlier ones (release the copy).
- **Event-driven updates** with a **transactional outbox** and **idempotent consumers**.
- **CQRS**: the search index is a read model rebuilt from events; the Borrowing service is the source of truth for availability at checkout.

2.5 *How would you achieve security in the proposed architecture?* 5 marks

- **Users:** login with OAuth2 / OpenID Connect at the gateway, which issues JWT tokens. Roles: member, librarian, admin (only librarians add or update books).
- **Service to service:** mutual TLS (service mesh), short-lived tokens, each service only allowed to call what it needs.
- **Data:** HTTPS everywhere, personal data and borrowing history encrypted at rest, secrets in a vault, least-privilege DB users.
- **Attacks:** rate limiting on search, input validation, a web application firewall, audit logs.

24 2024 · Question 3

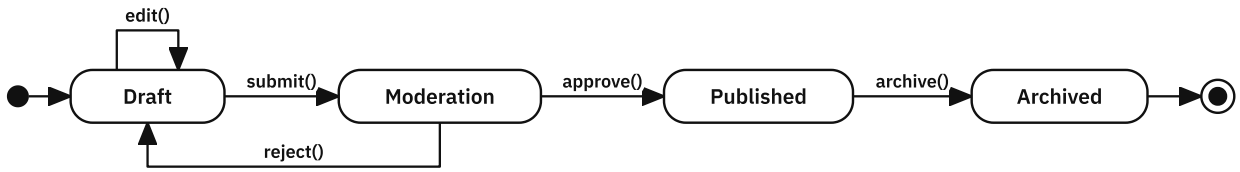
Document workflow: Draft → Moderation → Published → Archived.

STATE
25 MARKS

QUESTION 3 · 25 MARKS

Draft: created and editable; can move to Moderation. **Moderation:** under review; approved → Published, or rejected → back to Draft. **Published:** final, can't be edited; can only move to Archived. **Archived:** stored permanently, no further transitions.

3.1 *Define the state diagram for the above scenario.* 5 marks



Start dot, a rounded box per state, every arrow labelled with its event, and a bullseye after Archived because it is final.

3.2 *What is the best design pattern for this scenario? Provide reasons.* 5 marks

The **State** pattern (behavioural). It allows an object to change its behaviour when its internal state changes; the object appears to change its class.

Reasons: (1) the document has a **fixed set of four states**, and the **same action behaves differently** in each: `edit()` works in Draft but is refused in Published, `approve()` only works in Moderation; (2) without the pattern, every method would need an if/else on a status field, growing with each state; (3) each state class holds its own rules and decides the **next state**, so illegal moves (Archived → Draft) are refused cleanly; (4) a new state (e.g. "Under Legal Review") is a new class, and nothing else changes (Open/Closed).

3.4 *Explain the use of interfaces (OOP concept) in the above design.* 5 marks

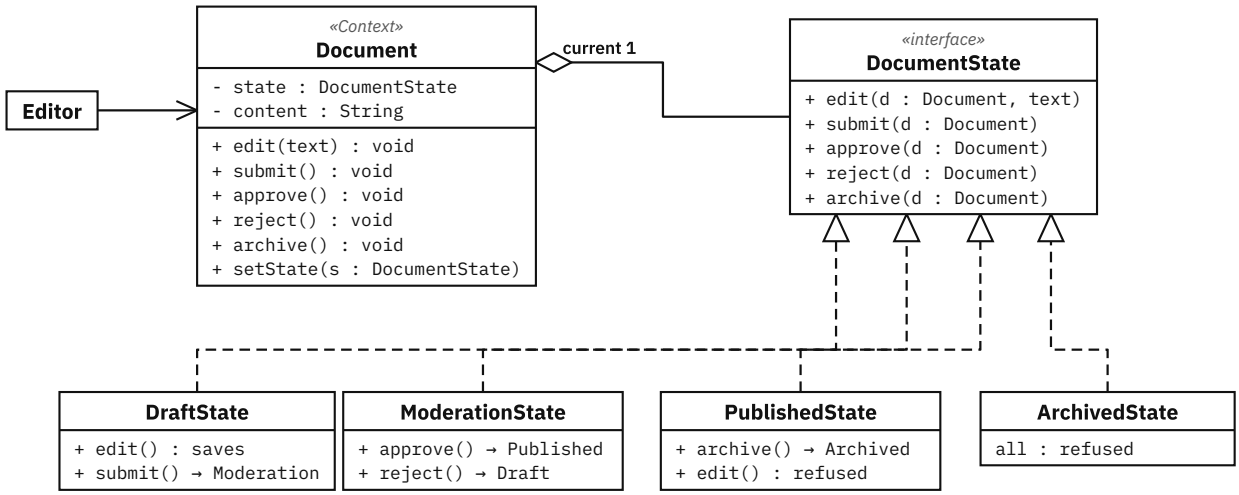
- `DocumentState` is a **contract**: every state must implement every event (`edit`, `submit`, `approve`, `reject`, `archive`), so nothing is forgotten.
- `Document` holds a `DocumentState` reference and calls `state.submit(this)` without knowing which state it is: **polymorphism**, programming to an interface.
- New states plug in without changing `Document` (**Open/Closed**, **Dependency Inversion**); states can be swapped at runtime and mocked in tests.

24 2024 · Question 3 (cont.)

The 10-mark class diagram, with code that matches it.

STATE
10 MARKS

3.3 *Create a class diagram illustrating the use of the identified design pattern in the workflow system.* 10 marks



For 10 marks show: the context with its state field and `setState()`, the interface with every event, all four concrete states, realisation arrows (dashed + hollow triangle), the has-a diamond on the context, and the client.

```
public interface DocumentState {
  void edit(Document d, String text);
  void submit(Document d);
  void approve(Document d);
  void reject(Document d);
  void archive(Document d);
}
public class Document {
  private DocumentState state =
      new DraftState();
  void setState(DocumentState s) {
    state = s; }
  public void submit() {
    state.submit(this); }
  // edit, approve, reject, archive:
  // same, each delegates to state
}
```

```
public class ModerationState
        implements DocumentState {
  public void approve(Document d) {
    d.setState(new PublishedState()); }
  public void reject(Document d) {
    d.setState(new DraftState()); }
  public void edit(Document d, String t) {
    System.out.println("Under review"); }
  public void submit(Document d) { }
  public void archive(Document d) {
    System.out.println("Publish first"); }
}
// PublishedState.edit():
//   "Published documents can't be edited"
// ArchivedState: every method refuses
```

2024 · Question 4

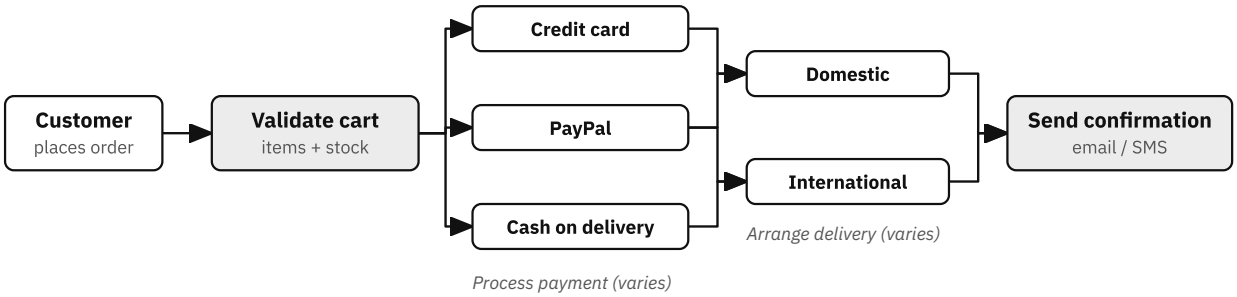
24 E-commerce checkout: standard steps, exceptions, abstract classes, authorization levels.

TEMPLATE · CHAIN · PROXY
25 MARKS

QUESTION 4 · 25 MARKS

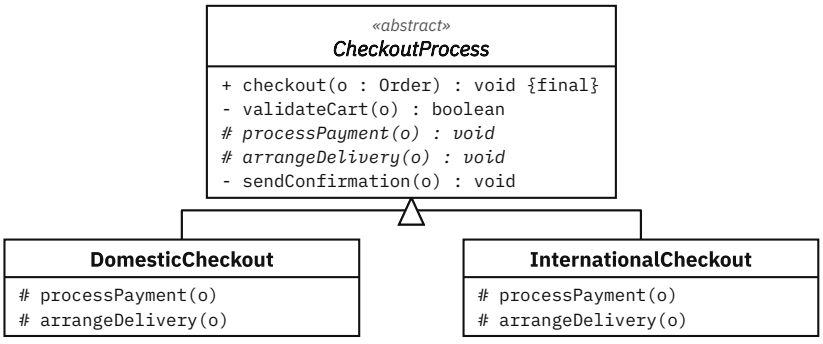
A standard checkout process: **Validate cart** (valid items, enough stock) → **Process payment** (credit card, PayPal, cash on delivery) → **Arrange delivery** (varies: domestic or international) → **Send confirmation** (email or SMS).

4.1 *Draw a diagram to depict the above scenario.* 5 marks



4.2 *What is the best design pattern for this case? Explain.* 5 marks

The **Template Method** pattern (behavioural). The checkout always has the same four steps in the same order; only payment and delivery differ. An abstract `CheckoutProcess` has a final `checkout(order)` that runs validate cart → process payment → arrange delivery → send confirmation. Validate cart and send confirmation are written once; `processPayment()` and `arrangeDelivery()` are abstract and implemented by subclasses such as `DomesticCheckout` and `InternationalCheckout`. **Reasons:** the order is enforced for every order, common steps aren't duplicated, and a new delivery type is just a subclass. (To avoid a subclass for every payment × delivery pair, the payment can be a separate object created by a Factory.)



24 2024 · Question 4 (cont.)

Exceptions, abstract classes and authorization levels.

CHAIN · TEMPLATE · PROXY
15 MARKS

4.3 *An exception-handling system is to be included. How would you handle the different types of exceptions?* 5 marks

Use **Chain of Responsibility**. Each handler deals with one type of exception and passes the rest on: `OutOfStockHandler` (tell the customer, suggest alternatives) → `PaymentFailedHandler` (retry or offer another method) → `DeliveryErrorHandler` (re-schedule, notify the courier) → `SystemErrorHandler` (log, alert the admin, show a friendly error). Ordered by how common and how severe they are. **Reasons:** the checkout code doesn't need to know who handles each error; a new type of exception is a new handler; it works like try/catch.

```
abstract class CheckoutErrorHandler {
  protected CheckoutErrorHandler next;
  public void handle(CheckoutException e) {
    if (canHandle(e)) resolve(e);
    else if (next != null) next.handle(e);
  }
  abstract boolean canHandle(
      CheckoutException e);
  abstract void resolve(
      CheckoutException e);
}
```

4.4 *How can abstract classes be used in the above scenario?* 5 marks

- `CheckoutProcess` is **abstract** because it must hold **real code** (the fixed order in the final `checkout()`, and the common steps validate cart and send confirmation) **and** declare **abstract steps** (`processPayment()`, `arrangeDelivery()`) that each subclass must write.
- It **can't be instantiated** on its own, because a "general" checkout makes no sense; only `DomesticCheckout` or `InternationalCheckout` can be created.
- An interface couldn't do this: it can't enforce the order or share the common code. Subclasses are used through the abstract type (**polymorphism**), and `final` stops them changing the order.
- `CheckoutErrorHandler` in 4.3 is also abstract: it holds the shared `next` link and passing logic.

4.5 *How would you incorporate different levels of authorization?* 5 marks

Protection Proxy. Sensitive operations (refunds, price changes, viewing all orders) sit behind proxies with the same interface; the proxy checks the user's role (customer, support agent, manager, admin) before forwarding, and logs the access. If an action needs approval that escalates with the amount (support can refund up to 5,000, a manager up to 50,000, above that an admin), use a **Chain of Responsibility** of approvers, each handling its level and passing larger requests up. Roles come from the JWT token issued at login (RBAC).

2025 · Question 1

25 Supermarket self-checkout kiosk: states, add-ons and one global tax calculator.

STATE · DECORATOR · SINGLETON
25 MARKS

QUESTION 1 · 25 MARKS

A supermarket introduces a self-checkout kiosk. Customers scan items, apply optional add-ons (gift-wrap, eco-bag, express packing) and pay. The kiosk operates in the states Idle, Scanning, Payment and Completed. An item's price can have dynamic add-ons without subclasses for every combination. A single global tax calculator must be used across all checkout counters for consistent tax.

1.1 *Identify suitable design patterns with reasons for: (a) managing kiosk states, (b) applying optional add-ons and calculating the final price dynamically, (c) ensuring only one tax calculator instance.* 6 marks

(a) kiosk states	**State**	The kiosk behaves differently in each state (scanning is ignored when Completed, paying is refused when Idle). Each state is its own class that handles the actions and decides the next state, so there is no if/else on a status flag, and a new state is just a new class.
(b) add-ons	**Decorator**	Each add-on wraps the item and adds its own cost and description, so any combination is built at runtime without 2^n subclasses. A new add-on is a new class (Open/Closed).
(c) one tax calculator	**Singleton**	A private constructor and a static `getInstance()` give one shared instance, so every counter uses the same rate and rules, and a rate change applies everywhere at once.

1.2 *Define an interface CheckoutItem with getCost() and getDescription().* 5 marks

```
// Component: items and add-ons
// share this interface
public interface CheckoutItem {
    double getCost();
    String getDescription();
}
```

1.3 *Create a concrete class GroceryItem implementing CheckoutItem.* 5 marks

```
public class GroceryItem
        implements CheckoutItem {
    private final String name;
    private final double price;
    public GroceryItem(String name,
                       double price) {
        this.name = name;
        this.price = price;
    }
    public double getCost() {
        return price; }
    public String getDescription() {
        return name; }
}
```

An add-on decorator

shows 1.2 and 1.3 in use, worth adding

```
public abstract class AddOnDecorator implements CheckoutItem {
    protected final CheckoutItem item;                  // the wrapped item
    public AddOnDecorator(CheckoutItem i) { item = i; }
}
public class GiftWrap extends AddOnDecorator {         // EcoBag, ExpressPacking: same shape
    public GiftWrap(CheckoutItem i) { super(i); }
    public double getCost()        { return item.getCost() + 150; }
    public String getDescription() { return item.getDescription() + " + gift wrap"; }
}
CheckoutItem it = new GiftWrap(new EcoBag(new GroceryItem("Rice 5kg", 1200)));  // 1200+50+150
```

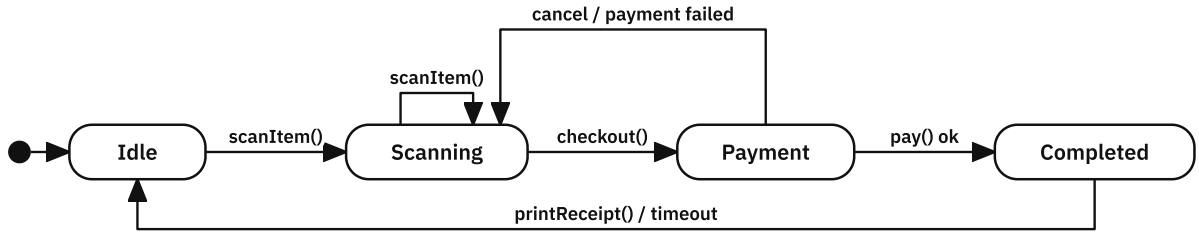
25 2025 · Question 1 (cont.)

Kiosk states with an example transition, and the Singleton in the pricing flow.

STATE · SINGLETON
9 MARKS

1.4 *Explain how you would incorporate the kiosk states (Idle → Scanning → Payment → Completed) using a design pattern. Provide an example transition.* 5 marks

Use the **State** pattern. `Kiosk` is the context and holds a `KioskState`. `IdleState`, `ScanningState`, `PaymentState` and `CompletedState` implement `scanItem()`, `checkout()`, `pay()` and `cancel()`. The kiosk passes every button press to its current state, and the state calls `kiosk.setState(next)` to move on.



Example transition: in `ScanningState`, the customer presses Checkout. `checkout()` adds up the items' `getCost()`, adds tax through the Singleton, stores the total and calls `kiosk.setState(new PaymentState())`. If the customer presses Pay while still Idle, `IdleState.pay()` refuses: "scan items first".

```
public class ScanningState implements KioskState {
    public void scanItem(Kiosk k, CheckoutItem i) { k.addItem(i); }      // stay
    public void checkout(Kiosk k) {
        double total = TaxCalculator.getInstance().applyTax(k.subtotal());
        k.setTotal(total);
        k.setState(new PaymentState());                                  // transition
    }
    public void pay(Kiosk k, double amt) { System.out.println("Press checkout first"); }
}
```

1.5 *Describe how the Singleton tax calculator can be integrated into the pricing flow.* 4 marks

Each scanned item is wrapped with its add-ons, so `item.getCost()` is already the final item price. When the kiosk moves to the Payment state it adds up the item costs and calls `TaxCalculator.getInstance().applyTax(subtotal)`. Because every counter calls `getInstance()`, they all get the **same** calculator with the same rate and rules, so tax is consistent across the store. When the rate changes, it is updated once (from configuration) and every counter uses it immediately.

```
public final class TaxCalculator {
  private static final TaxCalculator INSTANCE
      = new TaxCalculator();
  private double rate = Config.get("vat");
  private TaxCalculator() { }
  public static TaxCalculator getInstance() {
    return INSTANCE; }
  public double applyTax(double subtotal) {
    return subtotal * (1 + rate); }
}
```

25 2025 · Question 2

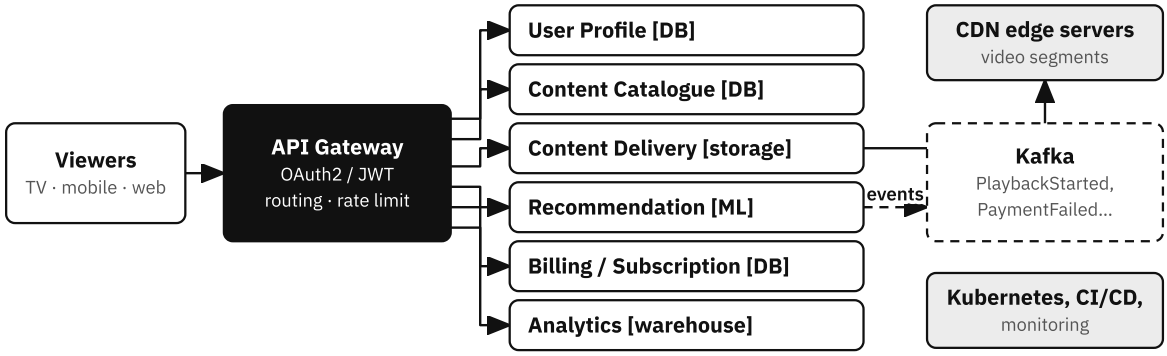
Media streaming platform on microservices.

MICROSERVICES
25 MARKS

QUESTION 2 · 25 MARKS

A media streaming platform wants a scalable, resilient microservices system. **User profile management** (register, accounts, preferences, viewing history, profiles) · **Content delivery** (store, transcode and stream video with a CDN) · **Recommendations** (personalised, ML/AI models) · **Billing & subscriptions** (plans, payments, invoices, renewals) · **Analytics** (streams, playback errors, engagement, insights). Each microservice should run independently while the user experience stays seamless.

2.1 *Draw a diagram to depict the microservices scenario.* 5 marks



2.2 *How can the organisation implement and manage microservices to achieve scalability, fault isolation and continuous deployment for this platform?* 5 marks

Scalability	Scale each service on its own. **Content Delivery** carries most of the load at peak evening hours, so it auto-scales and serves video from **CDN edge servers**; transcoding runs as a scalable worker queue; Recommendation runs on GPU nodes; Billing stays small. Stateless services on Kubernetes.
Fault isolation	Each service has its own process and database. If Recommendations fails, the home page shows a "popular now" list (graceful degradation); if Analytics fails, playback continues and events wait in the queue. Circuit breakers, timeouts, retries and bulkheads.
Continuous deployment	One repository and CI/CD pipeline per service, automated tests, Docker images, canary releases (new recommendation model to 5% of users first), versioned APIs, one team per service.
Manage it	API gateway, service registry, config server, and central logging, metrics and tracing to spot playback errors fast.

25 2025 · Question 2 (cont.)

Architecture, data consistency and security for the streaming platform.

MICROSERVICES
25 MARKS

2.3 *Propose a suitable microservices architecture for the platform.* 5 marks

SERVICE	RESPONSIBILITY	DATA / TECH
User Profile	accounts, profiles, preferences, viewing history	own DB
Content Catalogue	titles, metadata, rights by country	own DB + search index
Content Delivery	upload, transcode to several resolutions, stream	object storage + CDN
Recommendation	personalised lists from ML models	feature store, GPU nodes
Billing & Subscription	plans, payments, invoices, renewals	own DB, payment gateway
Analytics	streams, errors, engagement, insights	event stream → data warehouse
Notification	renewal reminders, new releases	consumes events

Clients (TV, mobile, web) go through an **API gateway**. Services use REST/gRPC for direct requests ("is this subscription active?") and publish events to **Kafka** ("PlaybackStarted", "PaymentFailed") for Analytics, Recommendation and Notification. Containers on Kubernetes, with a registry, config and monitoring.

2.4 *What methods can be used to maintain data consistency across the microservices?* 5 marks

- **Database per service** with **eventual consistency**: viewing history and recommendations can catch up a few seconds later.
- **Saga** for subscribing: create subscription → charge payment → activate the profile's plan. If the payment fails, a **compensating action** cancels the subscription.
- **Event-driven updates** with a **transactional outbox** and **idempotent consumers**, so a repeated "PaymentSucceeded" doesn't extend the plan twice.
- **CQRS** read models for analytics dashboards; strong consistency only where it matters (billing).

2.5 *How would you ensure security of user data and of service-to-service communication?* 5 marks

- **Users:** OAuth2 / OpenID Connect login at the gateway with JWT tokens; roles (viewer, profile owner, admin); limits on devices per account.
- **User data:** encrypted at rest and in transit (HTTPS), card data handled to **PCI-DSS** by the payment gateway, privacy rules for viewing history, secrets in a vault.
- **Service to service: mutual TLS** through a service mesh, short-lived service tokens, least-privilege access.
- **Content:** DRM and signed, expiring CDN URLs so streams can't be shared; rate limiting and a WAF.

2025 · Question 3

25 Library book-borrowing workflow: checks in order, book types, rare books, optional services.

CHAIN · FACTORY · PROXY · DECORATOR
25 MARKS

QUESTION 3 · 25 MARKS

When a user requests a book, the system checks in order: membership validity, book availability, pending fines. The checks must run in order and be easy to extend. Different categories of book (Fiction, Research, Magazine) must be created depending on the request. Rare books are in a restricted archive; users must not access them directly; access is controlled through permission checks. Optional services (Extended Borrow Period, SMS Notification, Email Receipt) should be applied without subclasses for each combination.

3.1 *Identify the design patterns, with reasons, for: (a) sequential validation checks, (b) creating different types of book, (c) controlling access to rare books, (d) adding optional services to the borrow transaction.* — 8 marks

(a) checks in order	**Chain of Responsibility**	Each check is a handler linked to the next; a failure stops the request with a reason. A new check is one class and one link, and the others don't change.
(b) book types	**Factory Method**	`BookFactory.createBook(type, title)` decides which class to create, so the client never uses `new` and new categories are easy to add.
(c) rare books	**Protection Proxy**	The proxy has the same interface as the archive and checks permissions (and logs access) before forwarding, so users can't reach the real archive directly.
(d) optional services	**Decorator**	Each service wraps the borrow transaction and adds its own behaviour or fee, in any combination, without a subclass per mix.

3.2 *Create a Book interface with getTitle() and getCategory().* — 2 marks

```
public interface Book {
    String getTitle();
    String getCategory();
}

public class FictionBook implements Book {
    private final String title;
    public FictionBook(String t) {
        title = t; }
    public String getTitle() {
        return title; }
    public String getCategory() {
        return "Fiction"; }
}
// ResearchBook, Magazine: same shape
```

3.3 *Write a simple BookFactory outline returning FictionBook, ResearchBook or Magazine.* — 5 marks

```
public class BookFactory {
  public static Book createBook(
          String type, String title) {
    switch (type.toUpperCase()) {
      case "FICTION":
        return new FictionBook(title);
      case "RESEARCH":
        return new ResearchBook(title);
      case "MAGAZINE":
        return new Magazine(title);
      default:
        throw new IllegalArgumentException(
            "Unknown type: " + type);
    }
  }
}
// client
Book b = BookFactory.createBook(
    "RESEARCH", "AI Ethics");
```

25 2025 · Question 3 (cont.)

The check chain and the rare-book proxy.

CHAIN · PROXY
10 MARKS

3.4 *Explain how you would handle the membership check → availability check → fine check.* 5 marks

With **Chain of Responsibility**. An abstract `BorrowCheck` holds a link to the next check. `MembershipCheck`, `AvailabilityCheck` and `FineCheck` each implement only their own test. The chain is built once in this order and every request goes to the first check. If a check fails, the chain stops and returns the reason ("membership expired", "book not available", "pending fine Rs. 200"); if all pass, the borrow goes ahead. A new rule (e.g. `BorrowLimitCheck`) is one new class added with one link.

```
public abstract class BorrowCheck {
  private BorrowCheck next;
  public BorrowCheck linkWith(BorrowCheck n) {
    next = n;
    return n;
  }
  public String check(Request r) {
    String err = test(r);
    if (err != null) return err;   // stop
    return next == null ? null
                        : next.check(r);
  }
  protected abstract String test(Request r);
}
```

```
class MembershipCheck extends BorrowCheck {
  protected String test(Request r) {
    return r.getUser().isMember() ? null
        : "Membership expired";
  }
}
// AvailabilityCheck, FineCheck: same shape

BorrowCheck chain = new MembershipCheck();
chain.linkWith(new AvailabilityCheck())
     .linkWith(new FineCheck());
String err = chain.check(request);
if (err == null) borrowService.issue(request);
else System.out.println(err);
```

3.5 *Explain the method(s) for controlling access to rare books and give one example.* 5 marks

A **Protection Proxy**. `ArchiveAccessProxy` and `RestrictedArchive` both implement `BookArchive`, and users are only ever given the proxy. Its `getBook(id)` first checks the user's role and permission (for example a researcher with an approved request and a valid membership), records the access in an audit log, and only then forwards to the real archive; otherwise it throws `AccessDeniedException`. The proxy can also make the access **read-only** or limit it to the reading room.

Example: a first-year student requests an 1820 manuscript: the proxy sees no approval and refuses. A researcher approved by the chief librarian requests it: the proxy allows it, logs "researcher R102 viewed MS-1820 at 10:42", and returns the book from the archive.

```
public RareBook getBook(String id) {
  if (!user.hasRole("RESEARCHER")
      || !user.isApproved(id))
    throw new
      AccessDeniedException("No permission");
  AuditLog.record(user, id);
  return archive.getBook(id);  // forward
}
```

(d) in code, if asked: `BorrowTransaction t = new EmailReceipt(new SmsNotification(new ExtendedPeriod(new BasicBorrow(book, user))));` Each decorator adds its fee or extra step and calls the transaction it wraps.

2025 · Question 4

25 E-commerce orders: payment workflow, automated checks, global order IDs, payment objects.

TEMPLATE · CHAIN · SINGLETON · FACTORY
25 MARKS

QUESTION 4 · 25 MARKS

Payment methods (credit card, PayPal, cash on delivery) follow a common workflow: validate payment details → process payment → send receipt; each method varies in step 2. The company wants to standardise the workflow while allowing custom behaviour per payment type. Orders go through automated checks (inventory, fraud, delivery-area validation) that must be maintainable and extendable. A single global OrderID generator must give every order a unique ID.

4.1 *Which design patterns fit? Valid reasons for: (a) payment workflow with common steps but different implementations, (b) sequential automated validations, (c) consistent global order IDs, (d) creating different payment objects.* — 8 marks

(a) payment workflow	**Template Method**	A final `handlePayment()` fixes validate → process → receipt; only `processPayment()` differs per method, so the workflow is standardised and the common steps aren't duplicated.
(b) validations	**Chain of Responsibility**	Inventory → fraud → delivery area as linked handlers; a failure stops the order; a new check is one new class and one link.
(c) order IDs	**Singleton**	One generator with one thread-safe counter, shared system-wide, so IDs are never duplicated.
(d) payment objects	**Factory Method**	`PaymentFactory.create(method)` returns the right processor from the customer's choice, so the client has no `new` or if-else.

4.2 *Define an abstract PaymentProcessor class (with the appropriate pattern) with validate(), processPayment(), sendReceipt() and handlePayment().* — 4 marks

```
public abstract class PaymentProcessor {
  // template method: fixed order
  public final void handlePayment(Order o) {
    o.setId(OrderIdGenerator
        .getInstance().nextId());
    if (!validate(o)) {
      System.out.println("Invalid");
      return;
    }
    processPayment(o);
    sendReceipt(o);
  }
  protected boolean validate(Order o) {
    return o.getAmount() > 0; }
  protected abstract void
      processPayment(Order o);
  protected void sendReceipt(Order o) {
    System.out.println("Receipt "
        + o.getId()); }
}
```

3.3 *Implement a concrete class CreditCardPaymentProcessor extending PaymentProcessor.* — 3 marks

```
public class CreditCardPaymentProcessor
        extends PaymentProcessor {

  protected boolean validate(Order o) {
    return super.validate(o)
        && CardValidator.isValid(
               o.getCardNo());
  }

  protected void processPayment(Order o) {
    System.out.println("Charging card");
    BankGateway.charge(o.getCardNo(),
                       o.getAmount());
  }
}
```

The paper numbers these as 3.3–3.5 by mistake; they belong to Question 4.

25 2025 · Question 4 (cont.)

The validation flow, the OrderID generator, and how all four patterns fit together.

CHAIN · SINGLETON
10 MARKS

3.4 *Briefly outline how a design pattern can be used for the validation flow (inventory → fraud → delivery area).* 5 marks

Chain of Responsibility (pipeline form). An abstract `Validator` holds `next`; `InventoryCheck`, `FraudCheck` and `DeliveryAreaCheck` each implement `check(order)`. The chain is built once in this order and the order service sends each order to the first validator. A failed check stops the chain and rejects the order with a reason; if all pass, payment starts. Cheap checks go first (inventory) and the expensive fraud scoring later. A new check (e.g. `AddressFormatCheck`) is one class and one link, and nothing else changes.

```
abstract class Validator {
  private Validator next;
  Validator linkWith(Validator n) {
    next = n; return n; }
  boolean validate(Order o) {
    if (!check(o)) return false;
    return next == null
        || next.validate(o);
  }
  abstract boolean check(Order o);
}
Validator v = new InventoryCheck();
v.linkWith(new FraudCheck())
 .linkWith(new DeliveryAreaCheck());
```

3.5 *Explain how you will design an OrderID generator and how it integrates with payment processing.* 5 marks

An **eager Singleton**: private constructor, one static final instance, and a thread-safe `AtomicLong` counter, so two orders placed at the same moment can never get the same ID. **Integration:** the first step of the template method `PaymentProcessor.handlePayment()` calls `OrderIdGenerator.getInstance().nextId()` and stores the ID on the order, then validate → process → send receipt (which prints the ID). Every payment type (card, PayPal, COD) inherits that step, so all share the one generator. For HA across several servers, the counter is backed by a database sequence.

```
public final class OrderIdGenerator {
  private static final OrderIdGenerator
      INSTANCE = new OrderIdGenerator();
  private final AtomicLong counter =
      new AtomicLong(1000);
  private OrderIdGenerator() { }
  public static OrderIdGenerator
      getInstance() { return INSTANCE; }
  public String nextId() {
    return "ORD-"
        + counter.incrementAndGet();
  }
}
```

How the four patterns fit together

draw it if you have time; it ties all the parts together

